Университет ИТМО

Факультет Программной Инженерии и Компьютерной техники

Информатика
Лабораторная работа №4
«Исследование протоколов, форматов обмена информацией и языков разметки документов»
Вариант №46

Выполнил: Джейранов Георгий Ксенофонтович
Группа P3119
Проверил: Осипов В.В

Санкт-Петербург
2026

Содержание

1……..……Содержание
2……..……Задание
3-4……Решение
5……..……Заключение
6…….Список используемой литературы

Задание

1. Определить номер варианта как остаток деления на 132 своего идентификационного номера в ISU: например, 125598 / 132 = 16. В случае, если в обоих указанных днях недели нет занятий, то увеличить номер варианта на восемь. В случае, если занятий нет и в новом наборе дней, то продолжать увеличивать на восемь.
2. Изучить форму Бэкуса-Наура.
3. Изучить основные принципы организации формальных грамматик.
4. Изучить особенности языков разметки/форматов JSON, RON, HCL, YAML, TOML, INI, XML.
5. Понять устройство страницы с расписанием на примере расписания лектора: https://itmo.ru/ru/schedule/3/125598/raspisanie_zanyatiy.htm
6. Исходя из структуры расписания конкретного дня, сформировать файл с расписанием в формате, указанном в задании в качестве исходного. При этом необходимо, чтобы хотя бы в одной из выбранных дней было не менее двух занятий (можно использовать своё персональное). В случае, если в данный день недели нет таких занятий, то увеличить номер варианта ещё на восемь.
7. Обязательное задание (позволяет набрать до 50 процентов от максимального числа баллов БаРС за данную лабораторную). Написать программу на языке Python 3.x или любом другом, которая: • осуществляет парсинг и конвертацию исходного файла в бинарный объект (=десериализацию); • для решения задачи использует формальные грамматики; то есть ваш код должен уметь осуществлять парсинг и конвертацию любых данных, представленных в исходном формате, в данные, представленные в результирующем формате (как с готовыми библиотеками из дополнительного задания №2); • не использовует готовые библиотеки, в том числе регулярные выражения в Python и библиотеки для загрузки файлов.
8. Дополнительное задание №1 (позволяет набрать +15 процентов от максимального числа баллов БаРС за данную лабораторную). Написать программу на языке Python 3.x или любом другом, которая: 1 • осуществляет парсинг и конвертацию бинароного объекта, полученного в обязательном задании, в новый формат (=сериализацию); • для решения задачи использует формальные грамматики; • не использовует готовые библиотеки, в том числе регулярные выражения в Python и библиотеки для загрузки файлов.
9. Дополнительное задание №2 (позволяет набрать +10 процентов от максимального числа баллов БаРС за данную лабораторную). a) Найти готовые библиотеки, осуществляющие аналогичный парсинг и конвертацию файлов (десериализацию и сериализацию). b) Переписать исходный код и код из дополнительного задания №1, применив найденные библиотеки. Регулярные выражения также нельзя использовать. c) Сравнить полученные результаты и объяснить их сходство/различие. Объяснение должно быть отражено в отчёте.
10.Дополнительное задание №3 (позволяет набрать +20 процентов от максимального числа баллов БаРС за данную лабораторную). Переписать код из дополнительного задания №1, чтобы серализация происводилась в XML файл.
11. Дополнительное задание №4 (позволяет набрать +5 процентов от максимального числа баллов БаРС за данную лабораторную). a) Используя свою исходную программу из обязательного задания и программы из дополнительных заданий, сравнить стократное время выполнения парсинга + конвертации в цикле. b) Проанализировать полученные результаты и объяснить их сходство/различие. Объяснение должно быть отражено в отчёте.
12.Проверить, что все пункты задания выполнены и выполнены верно.
13.Написать отчёт о проделанной работе. 14.Подготовиться к устным вопросам на защите.

Решение

Мой вариант:

№ варианта	Исходный формат	Результирующий формат	Дни недели
46	JSON	INI	Среда, суббота

Исходный файл schedule.json

Обязательное задание и дополнительное задание 1

С помощью файла deserializer.py мы делаем десериализацию файла schedule.json

Далее с помощью serializer.py мы делаем сериализацию полученного бинарнаного объекта в schedule.json

Исходный код:

deserializer.py

serializer.py

Результат:

schedule.ini

Дополнительное задание 2

Был создан файл JsonToIni.py который с использованием библиотек json и configparser делает конвертацию из json в ini

Исходный код:

JsonToIni.py

Результат:

schedule.ini

При сравнивании полученные ini файлы с использованием и без использования библиотек не отличаются, но можно заметить, что если в этой части кода не прописать счётчик, то в конечном ini файле будет описана только одна суббота вместо двух как в json файле. Это происходит так так в словарях (и в INI-файлах) не может быть двух секций с абсолютно одинаковым именем. у каждого ключа в словаре может быть только одно значение.

```
def convertWithLib():
    jsonTxt = open("schedule.json", "r", encoding="utf-8").read()
    data = json.loads(jsonTxt)
    config = configparser.ConfigParser()
    for key, itemlist in data.items():
        if isinstance(itemlist, list):
            index=1
            for itemdict in itemlist:
                secName = f"{key}_{index}"
                config[secName] = {}

                for k, v in itemdict.items():
                    if isinstance(v, list):
                        config[secName][k] = listToStr(v)
                    else:
                        config[secName][k] = str(v)
                index+=1
```

Дополнительное задание 3

Был взят бинарный объект из обязательного задания и переписан serializer.py

Исходный код:

serializer.py

Результат:

schedule.xml

Дополнительное задание 4
Я добавил свой код в данный цикл и с помощью библиотеки taim вычислил 100 кратное выполнение парсинга

```
import time
start_time = time.time()
for _ in range(100):
    МОЙ КОД
end_time = time.time()
print(f"Время выполнения 100 раз: {end_time - start_time} секунд")
```

Итог:
Обязательное задание + дополнительное 1 задание
Десериализация выполнялась 0.0850069522857666 секунд
Сериализация выполнялась 0.07858657836914062 секунд
Весь парсинг выполнялся 0.16359353065490722656 секунд
Исходный код:
deserializer.py
serializer.py

Дополнительное задание 2
Весь парсинг выполнялся 0.13263583183288574 секунд
Исходный код:
JsonToIni.py

Дополнительное задание 3
Десериализация выполнялась 0.10631966590881348 секунд
Сериализация выполнялась 0.07335662841796875 секунд
Весь парсинг выполнялся 0.1796762943267822265625 секунд
Исходный код:
deserializer.py
serializer.py

Можно заметить что ручной парсинг без использования библиотек выполнялся дольше, это обусловлено тем что библиотеки быстрее компилируются в машинный код. Так же видно, что парсинг в xml файл выполнялся чуть-чуть дольше чем в ini

Заключение

Я изучил основные принципы организации формальных грамматик, понял как устроены особенности языков разметки/форматов JSON, RON, HCL, YAML, TOML, INI, XML. А также понял, что такое форма-Бэкуса-Наура. Научился делать парсеры файлов на Python и понял как устроен сам процесс парсинга.

Список используемой литературы

Простая форма Бэкуса — Наура (БНФ)-https://divancoder.ru/2017/06/backus-naur-form/
Парсим на Python: Pyparsing для новичков-https://habr.com/ru/articles/239081/
Всё про конфиги-https://habr.com/ru/articles/866468/